

Introduction to DSA

Unit-1

A **data structure** refers to the way data is organized and stored in memory. It can be a simple primitive data type such as integer or float, or it can be a complex non-primitive data type like an array, linked list, stack, queue, tree, or graph. The choice of a data structure depends on the application requirements and the characteristics of the data being stored.

An **algorithm**, on the other hand, is a set of instructions that are used to solve a problem or perform a specific task. Algorithms can range from simple procedures like finding the maximum or minimum value in a list, to complex algorithms like sorting, searching, and graph traversal. The efficiency of an algorithm is measured in terms of its time and space complexity.

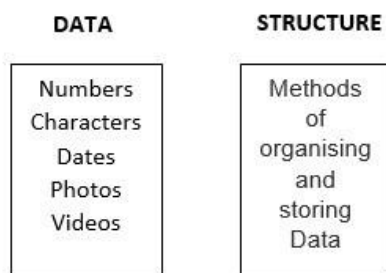
Data structures and algorithms are interrelated concepts, as the choice of a data structure can have a significant impact on the performance of an algorithm. For example, using an efficient data structure like a hash table can significantly improve the performance of a search algorithm.

1.Data Structure

The name "Data Structure" is a combination of two words: "Data" and "Structure". Let's go over each word individually:

Data: Data is information that must be processed and stored in a computer. Data can be anything from numbers to characters to dates to photos to videos.

Structure: Structure refers to how data is structured. It specifies the relationship between various data elements and how they are kept in memory. The structure has an impact on the efficiency of data operations.



Data structure can be defined as triple (D,F,A)

- D is data
- F is set of function and
- A is set of Axioms

1.1 NEED OF DATA STRUCTURES

- **Improved Time Complexity:** Using appropriate data structures can lead to better time complexity, making it possible to solve problems more quickly. For example, searching for an element in a sorted array is faster than searching for it in an unsorted array.
- **Better Space Complexity:** Data structures can help to reduce the amount of memory needed to store data. For example, using a linked list instead of an array can reduce the amount of memory needed to store the same data.
- **Efficient Data Retrieval:** Data structures make it easier to retrieve specific data efficiently. For example, a hash table can retrieve data in constant time, while searching through an unsorted array takes linear time.
- **Better Data Management:** Data structures make it easier to manage and manipulate data. For example, a stack can be used to implement an undo functionality in an application.
- **Solving Complex Problems:** Data structures can provide the foundation for efficient algorithms, making it possible to solve complex problems in a reasonable amount of time. For example, graph algorithms can be used to find the shortest path between two points or to find the minimum spanning tree of a graph.
- **Data Organization:** Data structures provide a way of organizing data in a meaningful and efficient manner, making it easier to access and manipulate data.
- **Data Retrieval Efficiency:** Data structures can provide fast and efficient retrieval of data, which is essential in many real-world applications.
- **Data Manipulation:** Data structures can provide efficient methods for adding, deleting, and modifying data, which is important in dynamic applications.
- **Code Reusability:** Reusable data structures can be used in many different applications, reducing the time and effort required to write and maintain code.
- **Problem-Solving Capability:** Data structures provide a way of modeling real-world problems and solving them in a more efficient and elegant manner.
- **Memory Requirements:** Data structures can be designed to use memory more efficiently, reducing the overall memory requirements of an application.
- **Data Security:** Data structures can be designed to provide additional security features, such as data encryption and protection against unauthorized access.

STATIC AND DYNAMIC DATA STRUCTURES

In Static data structure the size of the structure is fixed. The content of the data structure can be modified but without changing the memory space allocated to it. Static data structures, such as **arrays, have a fixed size** and are allocated at compile-time. This means that their memory size cannot be changed during program execution. Index-based access to elements is fast and efficient since the address of the element is known.

In Dynamic data structure the size of the structure is not fixed and can be modified during the operations performed on it. Dynamic data structures are designed to facilitate change of data structures in the run time. Dynamic data structures, on the other hand, have a variable size and are allocated at run-time. This means that their memory size can be changed during program execution. Memory can be dynamically allocated or deallocated during program execution. Due to this dynamic nature, accessing elements based on index may be slower as it may require memory allocation and deallocation. E.g. Linked List

OPERATIONS ON DATA STRUCTURES

This section discusses the different operations that can be performed on the various data structures previously mentioned.

- **Traversing** It means to access each data item exactly once so that it can be processed. For example, to print the names of all the students in a class. **Searching** It is used to find the location of one or more data items that satisfy the given constraint. Such a data item may or may not be present in the given collection of data items. For example, to find the names of all the students who secured 100 marks in mathematics.
- **Creation:** Process of creating a data structure
- **Inserting** It is used to add new data items to the given list of data items. For example, to add the details of a new student who has recently joined the course.
- **Deleting** It means to remove (delete) a particular data item from the given collection of data items. For example, to delete the name of a student who has left the course.
- **Sorting** Data items can be arranged in some order like ascending order or descending order depending on the type of application. For example, arranging the names of students in a class in an alphabetical order, or calculating the top three winners by arranging the scores in descending order and then extracting the top three.
- **Merging** Lists of two sorted data items can be combined to form a single list of sorted data items.

1.2 Characteristics and Types

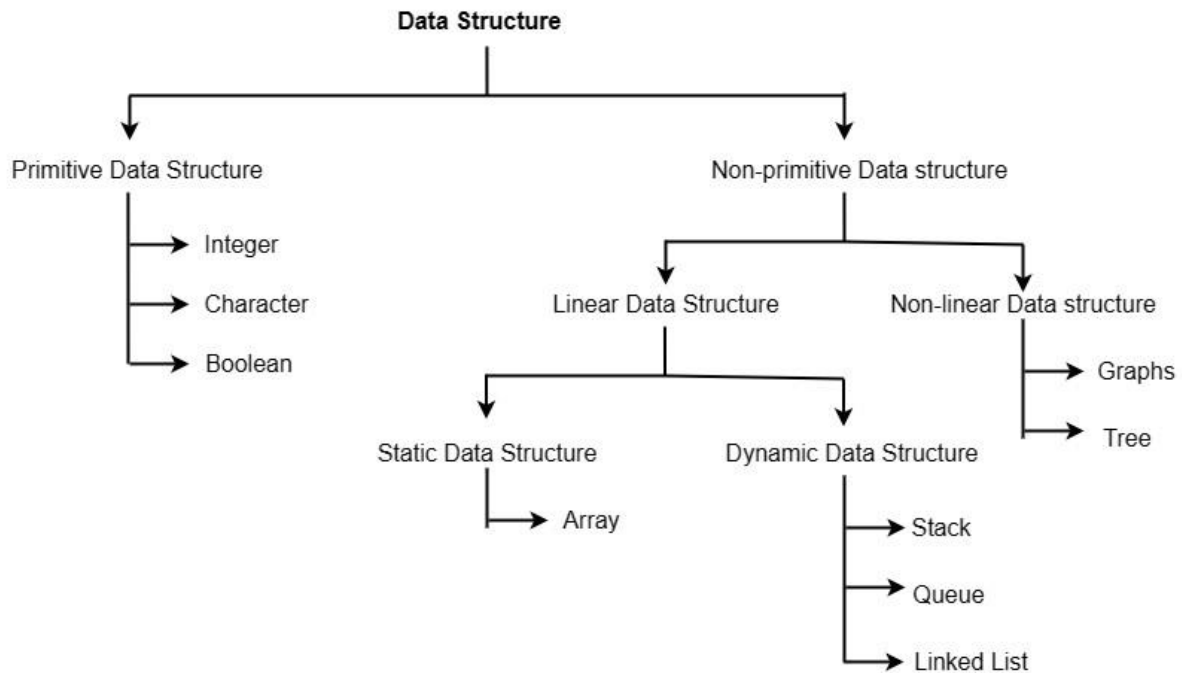
1.2.1 Characteristics of DS:-

1. **Linear vs. Non-Linear:** This refers to how the elements in the data structure are arranged. In linear data structures, elements are arranged in a sequential order, like a line. Examples include arrays, linked lists, stacks, and queues. Non-linear data structures, on the other hand, have a more complex arrangement of elements, where elements can be linked to each other in various ways. Examples include trees and graphs.
2. **Static vs. Dynamic:** Static data structures have a fixed size that is determined at compile time. This means that the amount of memory allocated for the data structure cannot be changed after it is created. Examples include arrays. Dynamic data structures, on the other hand, can grow or shrink in size at runtime. This allows them to accommodate changing data needs. Examples include linked lists and trees.
3. **Time Complexity:** This refers to the amount of time it takes to perform an operation on a data structure, typically measured in terms of the number of steps involved. Different data structures have different time complexities for different operations. For example, accessing an element in an array by its index is a constant time operation ($O(1)$), while searching for an element in a linked list can take linear time ($O(n)$) in the worst case.
4. **Space Complexity:** This refers to the amount of memory space required by a data structure to store its elements. It's important to consider the space complexity of a data structure in relation to the size of the data it stores. For example, arrays have a simple structure and low overhead, but they can waste space if the data size is not fixed. Linked

lists, on the other hand, can be more space-efficient but may have more overhead due to the additional pointers needed for each element.

5. **Correctness:** A data structure implementation should be correct and reliable. This means that it should accurately represent the data and perform operations according to its specifications.

1.2.2 Types (Classification) of DS:-



1. **Primitive data structures** are basic data types that are built into programming languages and are used to store simple values. These data types are generally predefined by the programming language and are directly supported by the computer hardware.

It is a basic data type that cannot be broken down into any other data types. Primitive data structures are typically used to store simple values, such as integers, characters, and Boolean values.

There are many different types of primitive data structures, but some of the most common include:

- **Integers:** Integers are whole numbers, such as 1, 2, 3, and so on.
- **Characters:** Characters are single letters, digits, or symbols, such as A, B, C, 1, 2, 3, and *.
- **Booleans:** Booleans are values that can be either true or false.
- **Floats:** Floats are numbers that can have a decimal point, such as 1.2, 3.4, and 5.6.

- **Doubles:** Doubles are similar to floats, but they can store more precise values.

2. **Non-primitive data structures** are the data structures that are created using primitive data structures. Non-primitive data types are used to represent complex structures and relationships that cannot be represented using primitive data types. It is a data structure that is created by the programmer using primitive data structures. Non-primitive data structures are typically used to store complex data, such as lists, trees, and graphs. They are fundamental to computer science and are used in various applications such as data storage, searching, and sorting algorithms, and artificial intelligence.

In linear data structures, the elements are arranged in sequence one after the other. Linear data structures are often used to store lists of data. They are also used to implement algorithms that operate on lists, such as sorting and searching.

One of the advantages of linear data structures is that they are easy to implement. They are also efficient in terms of space usage. However, they can be slow for certain operations, such as insertion and deletion. Some examples of linear data structures include arrays, linked lists, and stacks.

Linear Data are two types :-

1. Static Data structure
2. Dynamic Data structure

Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.

Dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.

Non-linear data structures are data structures that do not have a sequential order. Non-linear data structures are often used to store hierarchical data, such as the file system on a computer or the taxonomy of living things. They are also used to implement algorithms that operate on hierarchical data, such as searching and traversing.

One of the advantages of non-linear data structures is that they can be used to store data that would be difficult or impossible to store in a linear data structure. Some examples of non-linear data structures include trees and graphs.

- **Arrays:** It is a collection of elements of the same data type. Each element is identified by an index or a key.
- **Lists:** It is a collection of elements that can be of different data types. They can be accessed by their position in the list.
- **Stacks:** It is a data structure that follows the Last-In-First-Out (LIFO) principle. Elements are added and removed from the top of the stack.

- **Queues:** It is a data structure that follows the First-In-First-Out (FIFO) principle. Elements are added to the back of the queue and removed from the front.
- **Trees:** It is a hierarchical data structure that consists of nodes connected by edges. Each node has a parent and zero or more children.
- **Graphs:** It is a non-linear data structure that consists of vertices and edges. Vertices represent entities, and edges represent relationships between them.

2. Abstract Data Types (ADTs) and Data Structure

An ADT is a theoretical concept that describes a set of data values and the operations that can be performed on those values. It involves **what can be done with the data, not how has to be done.**

It is very similar to when we make a blueprint before actually getting into doing some job, be it constructing a computer or a building. The blueprint comprises all the minimum required logistics and the roadmap to pursuing the job. ADT is like a blueprint that tells minimal requirement along with some operations.

It is called “abstract” because it abstracts away (hide) the implementation details and focuses on what operations can be performed on the data type and what properties those operations should satisfy. Think it is like “black-box” where Developers can design and implement ADTs without worrying about how they will be used or by whom, and users can use ADTs without worrying about how they are implemented or how they work internally.

ADTs are like user defined data types which defines operation on values using function without specifying what is there inside the function and how the operations are performed.

Here are some key components and characteristics of Abstract Data Types:

Data Structure: An ADT is associated with a specific data structure that defines how data is organized, stored, and manipulated. Examples of data structures include arrays, linked lists, stacks, queues, trees, and graphs.

Operations: ADTs are defined by a set of operations or functions that can be performed on the data structure. These operations include basic actions like insertion, deletion, traversal, searching, and retrieval. The set of operations defines the behavior of the ADT, and the specific implementation details are abstracted away.

Encapsulation: ADTs encapsulate the internal details of the data structure, providing a clear interface for interacting with the data. Users of an ADT do not need to know how the operations are implemented; they only need to understand the provided interface.

Information Hiding: The internal details of an ADT are hidden from the user, promoting information hiding and reducing complexity. Users interact with the ADT through a welldefined set of methods, shielding them from the complexity of the underlying implementation.

Genericity:ADTs can be designed to work with different types of data, allowing for generality.For example, a stack ADT can be implemented to work with integers, characters, or any other data type without changing the basic stack operations.

Examples of ADTs:Stack:Operations: push, pop, isEmpty, isFull, peekQueue:Operations: enqueue, dequeue, isEmpty, isFull, frontList:Operations: insert, delete, find, iterateSet:Operations: add, remove, contains, union, intersectionMap (Dictionary):Operations: insert, delete, lookup, iterate

Implementation in Programming:ADTs are often implemented in programming languages using classes and interfaces.The interface defines the set of operations, while the class provides the actual implementation.Libraries and frameworks frequently use ADTs to provide reusable and abstracted data structures.

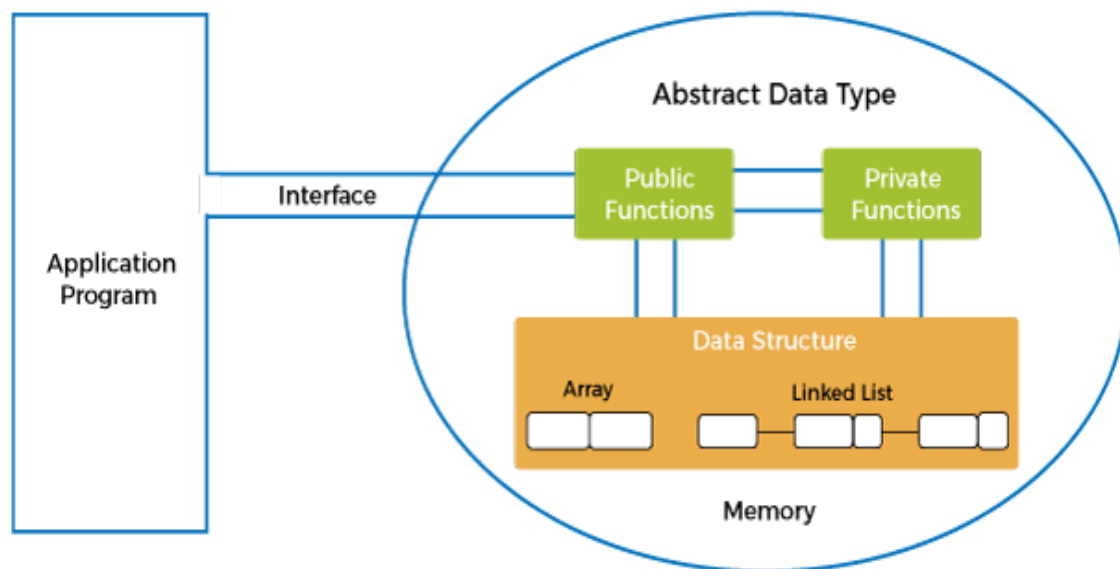
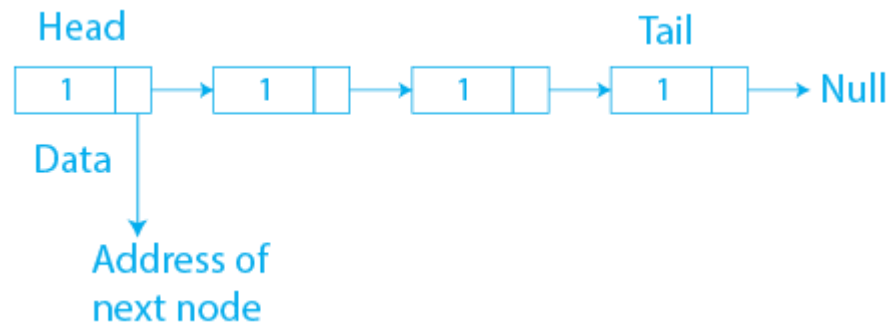


Fig:- ADTs model

Abstract data type model at level first encapsulation is performed then at second level abstraction is performed on the data structure. Abstraction means hiding internal details and showing only functionality. Encapsulation Combining the data and the member function in a single unit is considered encapsulation. All data that is being processed is maintained in a data structure and its implementation must not to be known to the user. (ADTs Structure). Data is inserted, deleted and updated through the application program via the interface. (ADTs Operation)

1. List ADT

Lists are linear data structures that hold data in a non-continuous structure. The list is made up of data storage containers known as "nodes." These nodes are linked to one another, which means that each node contains the address of another block.

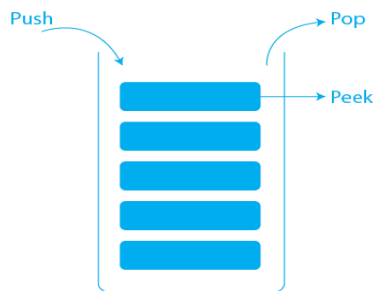


Some of the most essential operations defined in List ADT are listed below.

- `front()`: returns the value of the node present at the front of the list.
- `back()`: returns the value of the node present at the back of the list.
- `push_front(int val)`: creates a pointer with value = val and keeps this pointer to the front of the linked list.
- `push_back(int val)`: creates a pointer with value = val and keeps this pointer to the back of the linked list.
- `pop_front()`: removes the front node from the list.
- `pop_back()`: removes the last node from the list.
- `empty()`: returns true if the list is empty, otherwise returns false.
- `size()`: returns the number of nodes that are present in the list.

2. Stack ADT

A stack is a linear data structure that only allows data to be accessed from the top. It simply has two operations: push (to insert data to the top of the stack) and pop (to remove data from the stack). (used to remove data from the stack top).



- Some of the most essential operations defined in Stack ADT are listed below.
- `top()`: returns the value of the node present at the top of the stack.
- `push(int val)`: creates a node with value = val and puts it at the stack top.
- `pop()`: removes the node from the top of the stack.
- `empty()`: returns true if the stack is empty, otherwise returns false.
- `size()`: returns the number of nodes that are present in the stack.

3. Queue ADT

A queue is a linear data structure that allows data to be accessed from both ends. There are two main operations in the queue: push (this operation inserts data to the back of the queue) and pop (this operation is used to remove data from the front of the queue).

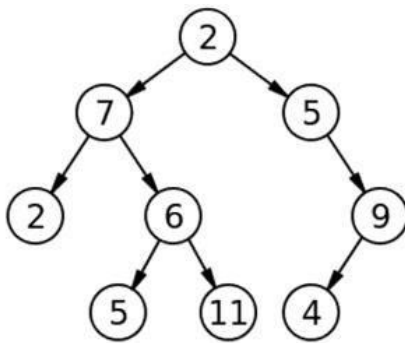


Some of the most essential operations defined in Queue ADT are listed below.

- `front()`: returns the value of the node present at the front of the queue.
- `back()`: returns the value of the node present at the back of the queue.
- `push(int val)`: creates a node with value = val and puts it at the front of the queue.
- `pop()`: removes the node from the rear of the queue.
- `empty()`: returns true if the queue is empty, otherwise returns false.
- `size()`: returns the number of nodes that are present in the queue.

4. Tree

A tree is a widely used abstract data type (ADT) or data structure implementing this ADT that simulates a hierarchical tree structure, with a root value and subtrees of children, represented as a set of linked nodes.



Comparison Between ADT and DS

- 
- Abstract data type a new type describes properties and operations
 - Each ADT operation is defined by its inputs and outputs
 - Collection of elements or we can say information.
 - Logical form
 - A data structure is the physical implementation of an ADT.
 - Each operation associated with the ADT is implemented by one or more subroutines in the implementation.
 - How collection is stored in memory
 - Physical/ Concrete form

Algorithm

Algorithm refers to a set of instructions or rules that are followed to solve a particular problem. Algorithms are used in computer programming to solve problems by breaking them down into smaller, more manageable steps that can be executed by a computer. It may involve various operations such as mathematical computations, logical decisions, comparisons, iterations, and data manipulation to achieve the desired output. **Characteristics of algorithm**

1. Unambiguous

This means that there is no room for interpretation or ambiguity in the instruction, the algorithm should be clear and precise, so that it can be implemented correctly and produce the desired output.

2. Input

The input is the data that is provided to the algorithm.

3. Output

The output is the result of the algorithm.

4. Effectiveness

Effectiveness refers to the ability of the algorithm to solve the problem in a reasonable amount of time and with a reasonable amount of resources. This includes considerations such as the running time of the algorithm and the amount of memory it uses.

1. Time Complexity

Time complexity is a measure of how long an algorithm takes to run. It is usually expressed as a function of the size of the input data.

2. Space Complexity

Time complexity is a measure of how long an algorithm takes to run. It is usually expressed as a function of the size of the input data.

5. Finiteness - For all different cases, the algorithm must produce result within a finite number of steps.

When designing an algorithm, it is important to consider the following factors:

- **The complexity of the problem:** The algorithm should be designed to be as simple as possible, while still being able to solve the problem.
- **The size of the input data:** The algorithm should be designed to be efficient, so that it can run quickly even with large input data.
- **The accuracy of the algorithm:** The algorithm should be designed to be accurate, so that it produces the correct results.

What is Performance Analysis of an algorithm?

When we have more than one algorithm to solve a problem (in computer science, there are multiple algorithms to solve a problem.), we need to select the best one. Performance analysis helps us to select the best algorithm from multiple algorithms to solve a problem.

When there are multiple alternative algorithms to solve a problem, we analyze them and pick the one which is best suitable for our requirements.

Performance of an algorithm is a process of making evaluative judgement about algorithms. In other word, Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

To compare algorithms, we use a set of parameters or set of elements like **memory required by that algorithm**, the **execution speed of that algorithm**, easy to understand, easy to implement, etc.,

When we want to analyze an algorithm, we consider only the space and time required by that particular algorithm and we ignore all the remaining elements.

Performance analysis of an algorithm is performed by using the following measures...

- Space required to complete the task of that algorithm (**Space Complexity**). It includes program space and data space
- Time required to complete the task of that algorithm (**Time Complexity**) **Algorithm**

Design

There are two approaches for designing an algorithm. these approaches include

1.Top-Down Approach :

2.Bottom-up approach

Top-Down Approach: In the top-down approach, a large problem is divided into small sub-problem. and keep repeating the process of decomposing problems until the complex problem is solved.

Bottom-up approach: The bottom-up approach is also known as the reverse of top-down approaches. In approach different, part of a complex program is solved using a programming language and then this is combined into a complete program.

Algorithm Design Techniques

Design Techniques (methods):-

1. Incremental Approach

The incremental approach is a problem-solving technique where a solution is built step-by-step by adding one component at a time, ensuring the partial solution remains valid or optimal at each step.

Steps:

1. **Start with a base case** (e.g., empty list or minimal structure).
2. **Add one component** (e.g., element, node, edge) at a time.
3. **Check for correctness** after each addition.
4. **Repeat until the full solution is built.**

e.g. Insertion sort

2. Divide and Conquer Approach: It is a top-down approach. The algorithms which follow the divide & conquer techniques involve three steps:

1. Divide the original problem into a set of subproblems.
 2. Solve every subproblem individually, recursively.
 3. Combine the solution of the subproblems (top level) into a solution of the whole original problem.
- Merge-Sort algorithm (using recursion)
 - Quick sort algorithm (using recursion)
 - Computing the length of the longest path in a binary tree (using recursion)
 - Computing Fibonacci numbers (using recursion)
 - Quick-hull

3. Greedy Technique: Greedy method is used to solve the optimization problem. An optimization problem is one in which we are given a set of input values, which are required either to be maximized or minimized (known as objective), i.e. some constraints or conditions.

Greedy Algorithm always makes the choice (greedy criteria) looks best at the moment, to optimize a given objective.

The greedy algorithm doesn't always guarantee the optimal solution however it generally produces a solution that is very close in value to the optimal.

Some examples of greedy algorithms are:

- Minimal spanning tree: Prim's algorithm, Kruskal's algorithm
- Dijkstra's algorithm for single-source shortest path problem
- Greedy algorithm for the Knapsack problem
- The coin exchange problem
- Huffman trees for optimal encoding

4. Backtracking Algorithm: Backtracking Algorithm tries each possibility until they find the right one. It is a depth-first search of the set of possible solution. During the search, if an alternative doesn't work, then backtrack to the choice point, the place which presented different alternatives, and tries the next alternative. Backtracking is a method in which a solution is found by searching through a large but finite number of states. With some pruning or bounding functions, we can narrow down our search.

For all problems (like NP-hard problems) for which there does not exist any other efficient algorithm, we use the backtracking algorithm.

E.g N-queens problems, suduko etc

Performance analysis and measurements

Time complexity

Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called time complexity. Time complexity is a measure of how long an algorithm takes to run.

Generally, the running time of an algorithm depends upon the following...

- Whether it is running on Single processor machine or Multi processor machine.
- Whether it is a 32 bit machine or 64 bit machine.
- Read and Write speed of the machine.
- The amount of time required by an algorithm to perform Arithmetic operations, logical operations, return value and assignment operations etc., □ Input data

When we calculate time complexity of an algorithm, **we consider only input data and ignore the remaining things**, as they are machine dependent. It is usually expressed as a function of the size of the input data. For example, an algorithm with time complexity $O(n)$ will take time proportional to the size of the input data.

Let us consider 2 developers created a program to sort n numbers and the time taken is as follow:

n	Running time on A	Running time on B
10	2 sec	~ 1 h
100	20 sec	~ 1.8 h
10^6	~ 55.5 h	~ 5.5 h
10^9	~ 6.3 years	~ 8.3 h

Which one is good?

Here, the algorithm by **A** works well for smaller number of elements. However, as we increase the number of elements, algorithm by **B** performs better.

Space Complexity

When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. Space complexity is the amount of memory required by an algorithm to solve a problem. It is also influenced by the input size.

Total amount of computer memory required by an algorithm to complete its execution is called as space complexity of that algorithm.

For any algorithm, memory is required for the following purposes...

- To store program instructions.
- To store constant values.
- To store variable values.
- And for few other things like function calls, jumping statements etc.,

To calculate the space complexity, we must know the memory required to store different datatype values (according to the compiler). For example, the C Programming Language compiler requires the following...

- 2 bytes to store Integer value.
- 1 byte to store Character value.

In general, the space complexity of an algorithm is related to the **amount of data that it needs to store in memory during its execution**. This data can be either input data, intermediate results, or auxiliary structures such as arrays or linked lists.

When analyzing an algorithm's time and space complexity, we often consider three different cases:

- **the best case, □ worst case, □ and average case.**

Best case: The best case represents the situation in which the algorithm performs the best or requires the minimum amount of time or space to complete its task. This scenario generally occurs when the input is already sorted, or the search element is found at the beginning of the list. The best-case time complexity of an algorithm is typically represented by the lower bound of the Big O notation.

Worst case: The worst case represents the situation in which the algorithm performs the worst or requires the maximum amount of time or space to complete its task. This scenario generally occurs when the input is in reverse order, or the search element is not present in the list. The worst-case time complexity of an algorithm is typically represented by the upper bound of the Big O notation.

Average case: The average case represents the scenario that is most likely to happen when running the algorithm on random input data. The average-case time complexity of an algorithm is typically represented by the expected time taken to complete the task, where the expectation is taken over all possible input data.

Rate of Growth

The pace at which the duration of execution expands in relation to the input is referred to as the rate of growth.

The rate of growth in algorithms refers to the efficiency with which an algorithm's execution time increases concerning the input size. In simpler terms, it analyzes how quickly an algorithm's performance deteriorates as the size of the data it processes expands. This metric is crucial in assessing the scalability and sustainability of algorithms in real-world applications.

let us Suppose you go to a store to buy cars and bicycles. If your friend meets you there and ask What you buy, then usually you say you want to buy a car. This is because the cost of the car is Compared to the cost of a bicycle (the cost of a bicycle is close to the cost of a bicycle) car).

Total Cost = Cost of Car + Cost of Bicycles

Total Cost = Cost of Car (Approximation)

For the above example, we can express the cost of the car and the cost of the bicycle In terms of function, for a given function, ignore relatively low-level items It doesn't matter (for larger input sizes, n).

For example, in the following cases, n^5 , $2n^3$, $100n^2$ And 100 is the personal cost of a certain function. Since n^5 is the highest, it is similar to n^5 growth rate.

$$n^5 + 2n^3 + 100n^2 + 100 = n^5 \text{ (Highest Growth Rate)}$$

Factors Influencing Rate of Growth:

Time Complexity: Time complexity is a fundamental aspect that directly impacts the rate of growth. Algorithms are often classified based on their time complexity, with constants, linear, logarithmic, quadratic, and exponential being common categories. Achieving lower time complexity signifies faster execution, making an algorithm more scalable.

Input Size: The rate of growth is inherently tied to the size of the input data. Different algorithms may exhibit varying growth rates based on the nature of their operations. Understanding how algorithms scale with increasing data size allows developers to choose the most suitable solution for a given problem.

Space Complexity: Apart from time complexity, the space complexity of an algorithm also contributes to its overall growth rate. Optimizing memory usage is vital for ensuring efficiency, particularly in resource constrained environments.

Rate of Growth	Name
1	Constant
Logn	Logarithmic
nlogn	Linear Logarithmic
n	Linear
n^2	Quadratic
n^3	Cubic
2^n	Exponential

Running Time Calculation

Asymptotic analysis

Asymptotic analysis is a way of analyzing the efficiency of an algorithm as a function of the size of its input. It is used to compare the efficiency of different algorithms for solving the same

problem. Asymptotic analysis provides a way to evaluate the performance of an algorithm by studying its behavior as the input size grows towards infinity.

Asymptotic notation is a mathematical notation that is used to represent the asymptotic growth rate of a function.

Asymptotic notation of an algorithm is a mathematical representation of its complexity.

Mathematical notation for algorithm analysis, particularly asymptotic notation, provides a standardized way to express the performance of an algorithm in terms of its time and space complexity. By using mathematical notation, we can precisely quantify the growth rate of an algorithm as the input size increases, making it easier to compare and evaluate different algorithms.

Here are some of the benefits of using asymptotic analysis:

- It can be used to compare the performance of different algorithms.
- It can be used to choose the best algorithm for a given problem.
- It can be used to improve the performance of algorithms.
- It can be used to understand the behavior of algorithms as their input sizes grow.

There are three main types of asymptotic notation:

- Big O notation (O)
- Omega notation (Ω)
- Theta notation (Θ)

Time Complexity or Time requirement

Let's suppose that each statement of the algorithm takes one unit of time. There are three statements in the given algorithm, but one statement consists of two operations add and assignment, so, total time take by the algorithm to execute is $1+(1+1)+1 = 4$ units of time. We can say that the time requirement of given algorithm is Order of 1, $O(1)$.

Space Complexity or Space Requirement

Here, in the given algorithm three variables: a, b, and s are declared. Each variable takes 1 unit of memory space, so total space (memory) requirement is $1 + 1 + 1 = 3$ units. It can be said that space requirement is of order 1, $O(1)$. If the time or space requirement for an algorithm is some constant like: 1, 5, 40 etc. then we say the time or space requirement is order of 1, $O(1)$.

```
function sum(int n,int[] Arr) {
```

```
int s = 0; int i;
```

```
----- 1 + 1 = 2 units
```

```
for(i=0;i<n;i++){
```

----- $1 + (n+1) + n = (2n + 2)$ units; we can ignore smaller terms. If $n = 4$ then the condition of the for loop will run: when $i=0$; $i = 1$; $i = 2$; $i = 3$; $i=4$; so, total number of times condition checking runs is $n + 1$.

```
s = s + Arr[i]; }
```

--n units (0, 1, 2, 3 if $n=4$) for displaying

```
display s;
}
```

--- 1 unit of time

The time complexity of the given algorithm which adds all the numbers of the given array of size n is $2 + (2n+2) + n + 1 = (3n + 5)$ Units. The degree of polynomial $2n+3$ is 1, so the algorithm is order of n , $O(n)$.

```
function sumArrays(int [] arra, int[] arrb){
```

```
int s = 0,i,j;
```

----- $1 + 1 + 1 = 3$ units

```
for(i=0;i<n;i++){
```

----- $n+1$ units (ignoring $i=0$; and $i++$)

```
for(j=0;j<n;j++){
```

----- $n * n+1$ units

```
s = arra[i] + arrb[j];
```

--n *n units for displaying

```
}
```

```
}
```

```
display s;
```

--- 1 unit of time

```
}
```

The time complexity of the given algorithm sumArrays, is: $3 + (n+1) + (n * (n+1)) + (n * n) + 1 = 2n^2+2n+5$ $2n^2+2n+5$ polynomial is of degree 2, so, the algorithm is of order of n^2 , $O(n^2)$ **Q)**

Why can't we calculate complexity in terms of seconds?

b) What do mean by algorithm analysis? Suppose the given algorithm and compute its total time $T(n)$ for worst case and average case.

Algorithm	Cost
for i = 1 to n	C1
for j = 1 to n-1	C2
printf("Pokhara University")	C3

$T(n) = (\text{Outer loop overhead}) + (\text{Inner loop overhead}) +$
 $(\text{Innermost statement cost})$

$$T(n) \approx (C1 \times n) + (C2 \times n(n - 1)) + (C3 \times n(n - 1))$$

$$T(n) = C1 \times n + (C2 + C3) \times n(n - 1)$$

$$T(n) = C1n + (C2 + C3)(n^2 - n)$$

$$T(n) = C1n + (C2 + C3)n^2 - (C2 + C3)n$$

$$T(n) = (C2 + C3)n^2 + (C1 - C2 - C3)n$$